COMP0005 Algorithms: Group Coursework Report

Group - 19 - Our Teams Final Report

Authors: *Mikael Toro, Evan Nguyen, Guillermo Arevalo Fernandez, Yajun He*

University College London

Contents

Chapter 1
Overview of Framework

1.1 Framework design and architecture

The experimental framework is designed to evaluate the performance of three search data structures: the Left-Leaning Red Black (LLRB) BST, 2-3 Tree, and AVL Tree. To guarantee a standardised comparison, all implementations inherit from an AbstractSearchInterface, which enforces identical signatures for insertElement and searchElement methods. As the sets are assumed to only grow over time, deletion operations are excluded from the scope of this implementation. The framework is built entirely from scratch, utilizing only the permitted Python libraries: timeit, random, string, and matplotlib.

1.2 Synthetic Data Generation

To thoroughly test the algorithms under varying conditions, a TestDataGenerator class generates synthetic string datasets of multiple sizes $\{1000, 5000, 10000, 25000, 50000\}$. To assess how content distribution impacts structural balance, the framework generates three distinct types of datasets:

Random Strings: 5-character alphanumeric strings to evaluate average-case performance.

Sorted Strings: Sequentially ordered strings to act as a stress test for the self-balancing mechanisms of each tree.

Prefix Strings: Strings sharing a 15-character common prefix followed by a random 5-character suffix, testing the computational overhead of expensive string comparisons.

1.3 Measurement Methodology

The ExperimentalFramework class isolates algorithmic execution time from data generation overhead. The timeit library is strictly utilized to measure only the execution of tree operations; the timer initiates only after the test data lists are fully constructed in memory and halts immediately upon completion of the insertions or searches.

Search performance is evaluated across two primary scenarios: successful searches (querying a sample of elements guaranteed to be in the tree) and unsuccessful searches (querying elements explicitly generated to be absent from the set). To minimize the impact of operating system background processes, the measurement is executed over five independent runs, computing the mean and standard deviation for each metric. Finally, an edge-case testing module is included to assess specific scenarios, such as the handling of duplicate insertions and the time discrepancy when searching for the first versus the last inserted element.

1.4 Hardware/software setup for our experiments

All of our groups experiments were carried out on a single machine to ensure that were received consistent and comparable results across all of our three data structures. The platform where we carried out our experiment was a home desktop PC having a CPU AMD Ryzen processor running at 4.7 GHz and 32 GB of RAM, and also running on Windows 11 Home (Build 26200). The code was executed using Python 3.11 within the Jupyter Notebook website. No other resource-intensive applications were running during the experiments to simply decrease/interference with any of our timing measurements.

Chapter 2
Performance results

2.1 2-3 Tree

A 2-3 Tree is a self-balancing tree in which each node has either one or two keys, with two or three child nodes respectively. All leaves sit at the same depth, which, theoretically, guarantees an $O(\log n)$ worst case complexity for per node search and insertion. The tree self-balances in insertion as all overflow is propagated upward through the middle key.

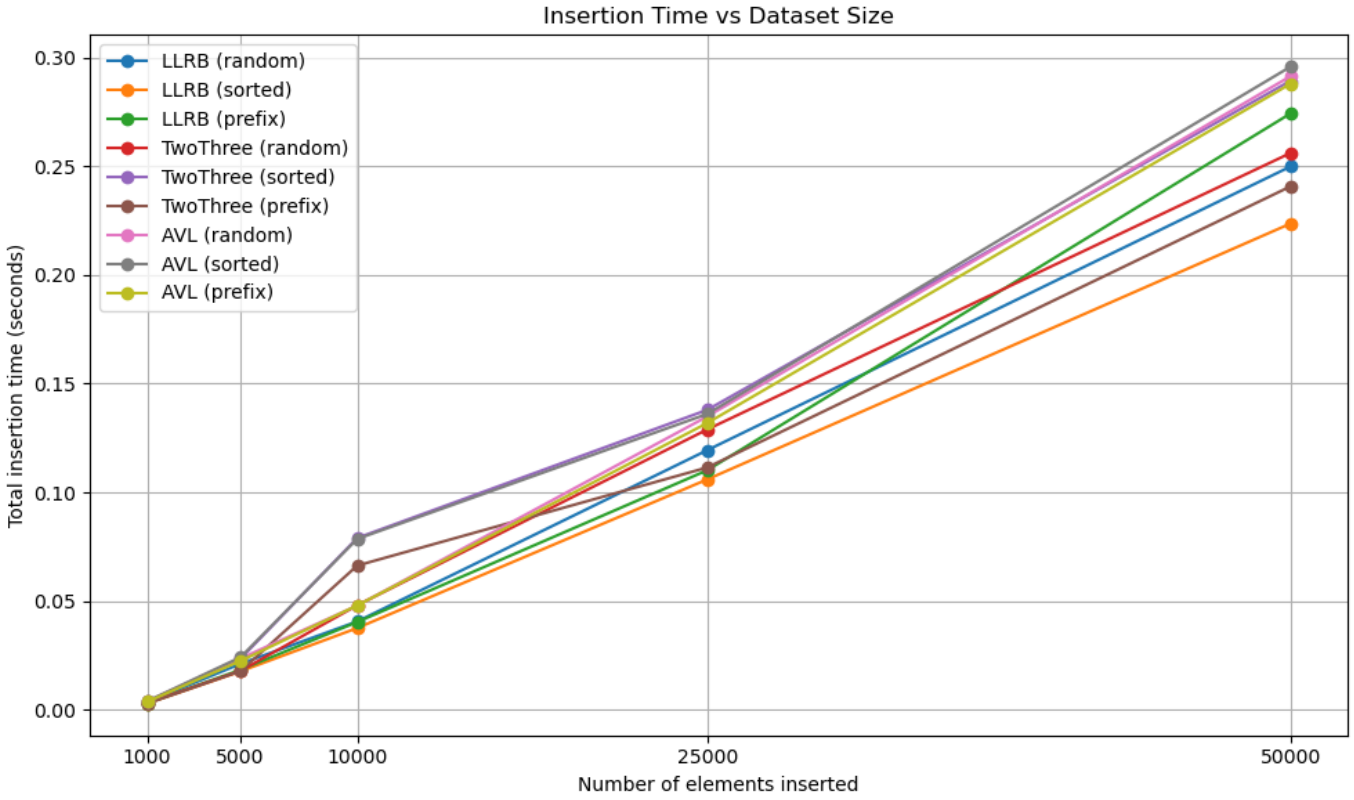

Figure 2.1: Insertion Time vs Dataset Size

Insertion Across all dataset sizes and distributions, insertion times scaled linearly with n and were consistent with an $O(n \log n)$ total insertion time. This is consistent with the theoretical worst-case complexity. Total insertion time of 50,000 elements was approximately 0.26 seconds (Figure 1), the second fastest. It's interesting to note the large disparity between time taken inserting random data and sorted data, one of the slowest insertions out of any in the experiment. This implies that constant rebalancing hurt this algorithm disproportionately.

Search The 2-3 tree consistently displayed the slowest search times of all three structures, across all conditions. At $n = 50{,}000$, the 2-3 tree's search time on random data was roughly 2.4 microseconds, which makes it roughly three times as slow compared to the LLRB and AVL trees (Figure 2). This, again, is very likely due to the specific implementation and per node overhead. Each node requires checking up to two keys, and Python list lookup carries more overhead than the standard scalar comparison used in binary tree structures.

Input Distribution Effects Sorted inputs consistently produced the worst-case performance for the 2-3 tree specifically, while the LLRB and AVL trees were not quite as affected. This is likely since given sorted insertions, every inserted key will travel the same rightmost path down the tree, resulting in frequent splits as inserted keys consistently overflow 3-nodes on that path.

Edge Cases Duplicate insertions were rejected in all cases. While there were some unexpectedly large disparities between searching for the first and last elements in the 2-3 tree when compared to LLRB and AVL, that disparity varied wildly between runs and is therefore inconclusive.

2.2 AVL Tree

In terms of the AVL tree its simply a self-balancing search tree where at every node it essentially keeps track/maintains a balance factor (which is the height of the left subtree minus the height of the right subtree) and this must stay within the range of -1 to $+1$. After each insertion the tree checks the balance factor and only when needed/necessary it performs rotation (left, right, left-right, or right-left) to restore this balance. What this ensures is that both insert and search operations always run in $O(\log n)$, even during the worst case.

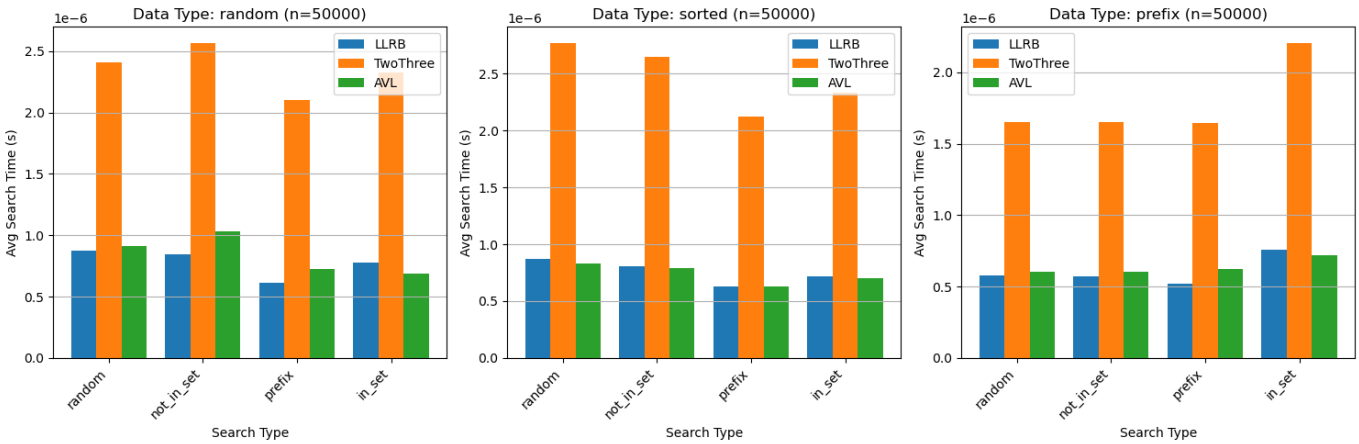

Figure 2.2: Insertion Time vs Dataset Size

Insertion Our results for random data is that the total insertion time increased from about $0.003s$ at $1,000$ elements to $0.29s$ at $50,000$ elements. With sorted data it performed almost the same as our random data showing that the AVL tree's rebalancing actually prevents the degenerate linear behaviour that a typical unbalanced BST would suffer from. When looking at prefix data it was slightly slower at smaller sizes but caught up/maintained the other distributions by $50,000$ elements. The time per insertion increased gradually with the

corresponding dataset size. On a log-log plot the growth appears as approximately a linear increase which matches our expected $O(n \log n)$ total cost for inserting n elements.

Search For our AVL search times they were consistently lower than our 2-3 Tree across all of our search types and data distributions and very close to LLRB. When searching for random elements in randomly inserted data, AVL's average search time increased from around 0.5 microseconds at 1,000 elements to about 0.9 microseconds at 50,000 elements, simply verifying/confirming our expected $O(\log n)$ growth. When looking at the bar charts for $n =$ 50,000 it clearly shows AVL and LLRB are performing nearly identically where both are significantly faster than the 2-3 Tree.

Edge cases For our duplicate insertions they were all correctly rejected (simply because all returned `False`) with a total time of 0.00025 s for 100 duplicate attempts. Searching for the first inserted element (3.08×10^{-7} s) was faster than searching for the last inserted element (5.71×10^{-7} s) because elements are simply distributed across different levels of the balanced tree

2.3 LLRBBST

The Left-Leaning Red Black Binary Search Tree (LLRBBST) is a self-balancing binary search tree. Each edge has a colour of either red or black, with the stipulation that all red edges go to the left. Its order is maintained through three operations: left rotation, right rotation, and colour flipping. With this, we can guarantee $O(\log n)$ worst case time complexity for both insert and search.

Insertion LLRB followed the theoretical time complexity of $O(n \log n)$ for total insertion time, as can be seen in the log-log plot. It had an average insert time of around 0.25 seconds for the tree of size 50,000 (Figure 1). It is the fastest out of the three for single item inserts as well, regardless of the size of the tree.

Search LLRB search was one of the faster searches along with AVL, with whom it shares an extremely similar performance. Given random data on a tree of size 50,000, the tree took on average 0.8 microseconds for a single element search (Figure 2). It was generally the fastest on random and prefix data.

Edge Cases Duplicate insertions were correctly rejected in all cases. Like the AVL, searching for the first inserted element on average took around half the time of searching for the last

inserted element. Although the trees balance themselves, these elements will sit at different depths. However, we should note this largely depends on the number of total insertions.

Comparative assessment

Chapter 3

All three structures share an $O(\log n)$ worst-case time complexity for both search and insertion, and $O(n)$ space complexity. Despite this theoretical equivalence, however, some practical features in implementation and the experimental results demonstrate significant differences in performance, and trade-offs with each structure.

Insertion Insertion performance was broadly comparable across all three data structures, with all three trees scaling consistently with $O(n \log n)$ total insertion time. The most notable difference was found under sorted input, where the 2-3 tree showed higher insertion times than both LLRB and AVL. This can be attributed to the split-heavy nature of sorted insertion in 2-3 trees.

Search Search performance showed the most significant differences between structures. The 2-3 tree was consistently the slowest across all input distributions and search types, in most cases running approximately three times slower than both LLRB and AVL trees at $n = 50{,}000$. However, the LLRB and AVL trees were largely indistinguishable in search performance across all conditions.

The $2-3$ tree's search inefficiency stems from overhead due to nodes having up to two keys to check, as well as the use of Python list lookup rather than simple scalar comparison. This is an implementation disadvantage rather than a theoretical one but would be important to note when choosing the best possible data structure.

Conclusion Based on the results, AVL trees appear to be the most suitable choice when consistent performance across all conditions is the priority. Its height balancing ensures the most predictable search and insertion times regardless of input order, which is represented in the edge case data.

LLRB trees were very often able to achieve faster results than AVL, however displayed notable deviations at low to mid data set sizes. While this is likely implementation specific, that would suggest that LLRB trees, while faster in the best case, risk inconsistency.

The $2-3$ tree performed the worst across all designed experiments. While theoretically equivalent to the other two, it does have practical limitations in implementation, as discussed prior. This may be less significant or not significant in other languages, and can theoretically be designed around, so our experimental results do not represent a conclusive verdict on the viability of $2-3$ trees.

Team Contributions

Chapter 4

Table 4.1: Group Contribution Summary

Student Name	**ID**	**Key Contributions**	**Share** [1]
Guillermo Arevalo Fernandez	25169757	Implemented the LLRB BST data structure, designed and built the Experimental Framework class including all timing, plotting, and statistical analysis methods and wrote our Section 2 performance analysis for the LLRB BST.	25%
Mikael Toro	24083417	Implemented the AVL Tree data structure (node class, insert with rotations, search) ran all the final experiments on my desktop PC to standardise our results across all three trees, wrote our Section 1.2 (hardware/software setup) and our Section 2 AVL Tree performance analysis and the overall complete final report in LaTeX.	25%
Evan Nguyen	24161696	Implemented the 2-3 Tree data structure, wrote Section 2 performance analysis for the 2-3 Tree, and authored Section 3 comparative assessment of all three data structures.	25%
Yajun He	25004885	Implemented the TestDataGenerator class for our synthetic data generation across the random, sorted, and prefix distributions and also wrote our Section 1 overview of the experimental framework design and architecture.	25%

[1] Percentage of total work. In a group of 4 with equal contribution, each member = 25%.